

在深度学习的微调过程(尤其是大模型微调)中,显存的消耗主要围绕着训练循环的三个核心步骤:前向传播 (**Forward Pass**)、反向传播 (**Backward Pass**) 和 权重更新 (**Optimization Step**)。根据课程资料中的显存估算公式:“微调时的总显存 = 模型权重显存 + 优化器状态显存 + 梯度显存 + 前向传播激活值显存”,我们可以把这四部分的具体使用场景拆解如下:

1. 模型权重显存 (Model Weights)

- 具体场景:这是大模型“大脑”本身的物理体积,包含了所有在预训练阶段学到的参数矩阵。
- 使用机制:在微调任务启动时,这部分数据会被最先加载并常驻于显卡的显存中。无论是进行前向传播计算输出,还是在反向传播中作为计算基准,显卡都需要随时读取这些权重。以 7B 模型为例,在 FP16(半精度)格式下,这部分雷打不动地需要大约 14GB 的显存。

2. 前向传播激活值显存 (Activations)

- 具体场景:当输入的数据(比如你喂给模型的训练语料)一层一层穿过神经网络时,每一层经过计算都会吐出一些“中间结果”,这就是激活值。
- 使用机制:这些激活值在前向传播时产生,并且必须暂存在显存中不能丢弃。为什么?因为在随后的反向传播阶段,计算误差链式法则时必须依赖这些中间结果。直到该层的反向传播计算完毕,这部分显存才会被释放。它的占用大小与你设置的 Batch Size(批次大小)和序列长度紧密相关,估算通常为模型权重的 20%-30%。

3. 梯度显存 (Gradients)

- 具体场景:梯度代表了“纠错指南”。它告诉模型,为了让当前的输出更接近标准答案,每一个参数应该往哪个方向、以多大的幅度进行微调。
- 使用机制:在反向传播阶段,系统会根据最终的误差和前面保存的“激活值”计算出梯度。每一个需要被训练的参数,都会在显存中生成一个对应的梯度值。如果你使用 LoRA 这种高效微调技术(假设可训练参数比例为 1%),由于绝大多数参数被冻结,系统只需要为那 1% 的参数生成梯度,因此极大地节省了显存。

4. 优化器状态显存 (Optimizer States)

- 具体场景:优化器(如业界常用的 AdamW)是负责执行权重更新的“教练”。为了让训练过程更平滑、收敛更快,它不能只看当前的梯度,还要记录每个参数的“历史更新轨迹”(通常是动量和方差)。
- 使用机制:在反向传播结束后的“权重更新”阶段发挥作用。以 AdamW 为例,它通常需要为每一个可训练参数额外保存两个历史状态变量。这就是全参数微调时显存爆炸的隐形元凶。但如果使用 LoRA,因为只训练极少量的参数,优化器状态和梯度加在一起总共也只占用大约 0.7GB 的显存。

理解了这四部分的运转机制,就能明白为什么像 LoRA 这种技术能如此成功:它巧妙地避开了去

动巨大的“模型权重”，只通过新增极小比例的旁路矩阵，就把最吃显存的“梯度”和“优化器状态”按比例砍掉了 90% 以上。

需要我为你进一步解释课程大纲中提到的 **Unslow** 框架 是如何通过底层优化在这四个环节中进行一步榨干显卡性能，做到速度提升 2-5 倍的吗？